

Expresiones regulares y autómatas

Repaso de la Nota 04 y trabajo con el Cuadernillo de ejercicios 03

Manuel Soto Romero Araceli Liliana Reyes Cabello

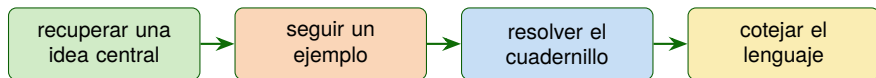
Colegio de Ciencia y Tecnología, UACM

Semestre 2026-2

Iniciamos con Kahoot

Iniciamos con el Kahoot

Ruta de trabajo



Durante la sesión

Alternaremos el repaso de la Nota 04 con los ejercicios del **Cuadernillo de ejercicios 03**.

En cada transformación preguntaremos: ¿cambió la representación o cambió el lenguaje?

Del patrón a un reconocedor



Especificar

Describe cuáles cadenas pertenecen a una categoría léxica.

Reconocer

Recibe una cadena y decide si pertenece al lenguaje especificado.

Las conversiones cambian la representación, pero conservan el lenguaje.

Alfabeto, cadena y lenguaje

Alfabeto Σ

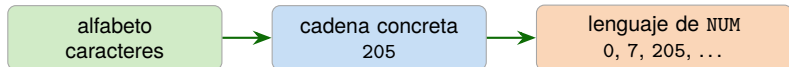
Conjunto finito de símbolos.

Cadena

Secuencia finita de símbolos; ε es la cadena vacía.

Lenguaje

Conjunto de cadenas formadas sobre un alfabeto.



205 es una cadena del lenguaje de NUM; no es el lenguaje completo.

Operaciones de las expresiones regulares

Operación	Notación	Qué cadenas describe
Unión	$r \mid s$	Las que pertenecen a $L(r)$, a $L(s)$ o a ambos.
Concatenación	rs	Las formadas al unir una cadena de $L(r)$ con una de $L(s)$.
Cerradura	r^*	Cero o más repeticiones; siempre incluye ε .
Repetición positiva	$r^+ = rr^*$	Una o más repeticiones; no incluye ε por sí sola.

Precedencia

cerradura > concatenación > unión

Leamos el lenguaje antes de probar cadenas

Expresión	Lenguaje denotado
$a \mid b$	$\{a, b\}$.
ab	Sólo la cadena ab .
a^*	$\{\varepsilon, a, aa, aaa, \dots\}$.
$(a \mid b)^*$	Todas las cadenas finitas sobre $\{a, b\}$, incluida ε .
$a(a \mid b)^*$	Cadenas no vacías sobre $\{a, b\}$ cuyo primer símbolo es a .

Pregunta de control: ¿qué exige la expresión y qué parte puede quedar vacía?

Trabajo con el cuadernillo

Ahora resolvemos el Cuadernillo 03

Ejercicio 1

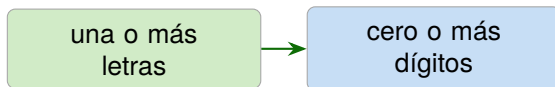
- ▶ **Ejercicio 1:** cotejar cadenas con varias expresiones regulares.
- ▶ Al justificar, identifica qué operación obliga a consumir símbolos y cuál permite ε .

Patrones variables del IMP básico

$letra = [A-Za-z],$
 $digito = [0-9],$
 $LOC = letra^+ digito^*,$
 $NUM = digito^+.$

Token	Sí satisface	No satisface
LOC	x, total, x20	20x, x2y, x_1
NUM	0, 25, 2048	ϵ , 2x, -5

Cómo leer LOC y NUM



- ▶ En $x2y$, una letra aparece después de comenzar la parte de dígitos.
- ▶ En $20x$, falta la parte inicial de una o más letras.
- ▶ En -5 , el signo no es un dígito: se reconocería como MINUS seguido de NUM(5).
- ▶ `while` satisface LOC y también es un lexema fijo de WHILE; la coordinación de reglas pertenece al tema 2.3.

Trabajo con el cuadernillo

Ahora resolvemos el Cuadernillo 03

Ejercicios 2 y 3

- ▶ **Ejercicio 2:** clasificar lexemas con los patrones aprobados de LOC y NUM.
- ▶ **Ejercicio 3:** encontrar un contraejemplo que haga visible el error de cada propuesta.

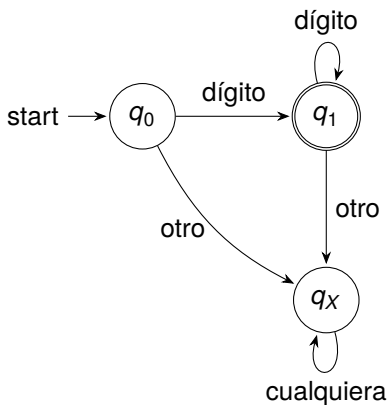
AFN y AFD: mismo poder, distinto recorrido

Aspecto	AFN	AFD
Siguiente estado	Puede ser un conjunto de estados.	Es un solo estado.
Transiciones ϵ	Se permiten.	No se permiten.
Aceptación	Basta un recorrido que acepte.	Se sigue el único recorrido.
Uso	Facilita componer subexpresiones.	Facilita reconocer símbolo por símbolo.

Equivalencia expresiva

Para cada AFN existe un AFD que reconoce el mismo lenguaje.

Ejemplo resuelto: AFD para NUM



Entrada 25

$q_0 \rightarrow q_1 \rightarrow q_1$: acepta.

Entrada 2x

$q_0 \rightarrow q_1 \rightarrow q_x$: rechaza.

Trabajo con el cuadernillo

Ahora resolvemos el Cuadernillo 03

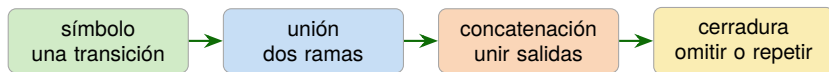
Ejercicios 4 y 5

- ▶ **Ejercicio 4:** distinguir el funcionamiento de un AFN y un AFD.
- ▶ **Ejercicio 5:** construir un AFD para LOC y trazar cuatro cadenas.

De una expresión regular a un AFN

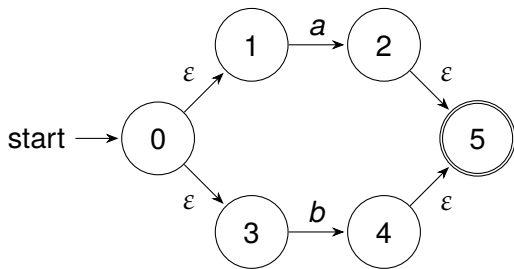
Método de Thompson

Construye un AFN para cada subexpresión y combina las piezas siguiendo la estructura de la expresión regular.



Las transiciones ε permiten conectar componentes sin consumir entrada.

Ejemplo resuelto: AFN de Thompson para $a \mid b$



- ▶ a : $0 \xrightarrow{\epsilon} 1 \xrightarrow{a} 2 \xrightarrow{\epsilon} 5$; acepta.
- ▶ b : sigue la rama inferior y acepta.
- ▶ ab : ninguna rama consume toda la entrada; rechaza.

Construcción de subconjuntos

Dos operaciones

cerradura- ϵ reúne los estados alcanzables sin consumir símbolos.

mover(T, a) reúne los estados alcanzables desde T al consumir a .

1. Inicia con $A_0 = \text{cerradura}_\epsilon(\{q_0\})$.
2. Para cada conjunto T y símbolo a , calcula

$$U = \text{cerradura}_\epsilon(\text{mover}(T, a)).$$

3. Agrega U sólo si todavía no se había descubierto.
4. Marca U como aceptación cuando $U \cap F \neq \emptyset$.
5. Repite hasta que no queden conjuntos pendientes.

El AFN de $a \mid b$ convertido a AFD

$$A = \{0, 1, 3\}, \quad B = \{2, 5\}, \\ C = \{4, 5\}, \quad D = \emptyset.$$

Estado	Con a	Con b	Acepta
A	B	C	No
B	D	D	Sí
C	D	D	Sí
D	D	D	No

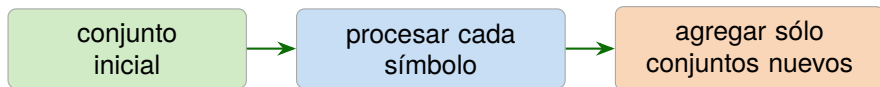
Un estado del AFD representa un conjunto de estados posibles del AFN.

Ahora resolvemos el Cuadernillo 03

Ejercicio 6

- ▶ Calcula las cerraduras y movimientos del AFN de $a \mid b$.
- ▶ Completa la tabla del AFD y marca los conjuntos de aceptación.
- ▶ Compara los $2^6 = 64$ subconjuntos posibles con los cuatro alcanzables.

Construimos sólo los subconjuntos alcanzables



Conjunto potencia y estados efectivos

Si el AFN tiene n estados, $\mathcal{P}(Q)$ contiene hasta 2^n subconjuntos. El AFD sólo necesita los que pueden alcanzarse desde su estado inicial.

Cada conjunto se procesa una vez; el procedimiento termina cuando no aparecen estados nuevos.

Trabajo con el cuadernillo

Ahora resolvemos el Cuadernillo 03

Ejercicio 7

- ▶ Aplica la construcción a un AFN sin transiciones ε .
- ▶ Agrega únicamente los subconjuntos alcanzables.
- ▶ Traza 1101 y describe el lenguaje reconocido.

Minimizar sin cambiar el lenguaje

Estados distinguibles

Existe una continuación que, al comenzar en ambos estados, termina en aceptación sólo desde uno.

Estados equivalentes

Ninguna continuación permite distinguirlos; pueden reunirse en el AFD mínimo.

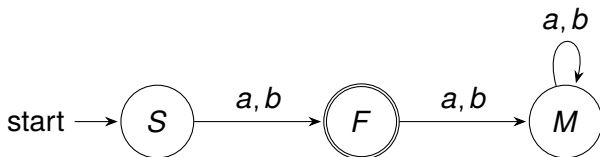
1. Separa estados de aceptación y de no aceptación.
2. Dentro de cada grupo, compara a qué grupos llegan las transiciones.
3. Separa los estados cuyo comportamiento difiere.
4. Repite hasta obtener una partición estable.

Ejemplo resuelto: minimizar el AFD de $a \mid b$

$$\Pi_0 = \{\{B, C\}, \{A, D\}\}$$

- ▶ B y C aceptan y, con a o b , llegan a D : permanecen juntos.
- ▶ A y D se distinguen con a : desde A se acepta y desde D no.

$$\Pi_f = \{\{B, C\}, \{A\}, \{D\}\}$$



Trabajo con el cuadernillo

Ahora resolvemos el Cuadernillo 03

Ejercicios 8 y 9

- ▶ **Ejercicio 8:** aplicar la tabla de pares al AFD de $a \mid b$.
- ▶ **Ejercicio 9:** refinar una partición hasta construir otro AFD mínimo.

Aplicación integrada a un fragmento de IMP

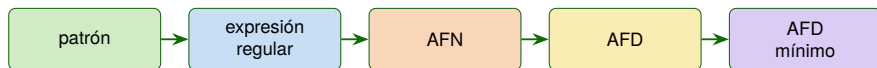
Entrada

```
if x20 <= 25 then x20 := x20 + 1 else skip
```

Lexema	Token	Patrón o lexema fijo
if	IF	Lexema fijo if.
x20	LOC(x20)	<i>letra⁺digito[*]</i> .
<=	LE	Lexema fijo <=.
25	NUM(25)	<i>digito⁺</i> .
:=	ASSIGN	Lexema fijo :=.
+	PLUS	Lexema fijo +.

```
IF LOC(x20) LE NUM(25) THEN LOC(x20) ASSIGN LOC(x20) PLUS NUM(1) ELSE SKIP
```

Qué hemos construido y qué falta



En el tema 2.3 todavía falta

- ▶ coordinar todas las reglas y resolver coincidencias;
- ▶ identificar el lexema adecuado dentro del flujo;
- ▶ omitir los espacios permitidos y reportar errores;
- ▶ expresar la especificación en LARK.

Ahora resolvemos el Cuadernillo 03

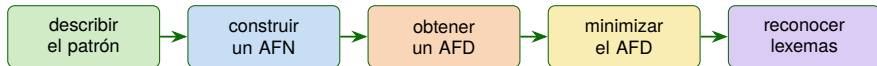
Ejercicio 10

- ▶ Separa los lexemas y escribe la secuencia conceptual de tokens.
- ▶ Relaciona x20 y 25 con sus patrones.
- ▶ Ordena la ruta del patrón al AFD mínimo.
- ▶ Delimita el trabajo que corresponde al tema 2.3.

Confusiones que debemos evitar

Confusión	Corrección
Una expresión regular es una cadena de ejemplo.	La expresión describe un lenguaje; una cadena puede pertenecer o no a él.
Un AFN acepta sólo si todos sus recorridos aceptan.	Basta que exista un recorrido que consuma toda la entrada y acepte.
Una transición ϵ consume un símbolo.	Cambia de estado sin consumir entrada.
Todo subconjunto posible aparece en el AFD.	Sólo se incorporan los subconjuntos alcanzables desde el inicio.
Minimizar puede cambiar las cadenas aceptadas.	El AFD mínimo debe reconocer exactamente el mismo lenguaje.

Conclusión



Idea de cierre

Las expresiones regulares especifican los lenguajes de los tokens y los autómatas permiten reconocer sus cadenas. Thompson, la construcción de subconjuntos y la minimización cambian la representación sin cambiar el lenguaje.

Ya contamos con la base formal para construir el analizador léxico de IMP.

Referencias

1. Soto Romero, M. y Reyes Cabello, A. L. (2026). *Programación de Sistemas. Nota de clase 04: Expresiones regulares y autómatas finitos*. Colegio de Ciencia y Tecnología, UACM.
2. Soto Romero, M. y Reyes Cabello, A. L. (2026). *Programación de Sistemas. Cuadernillo de ejercicios 03: Expresiones regulares y autómatas*. Colegio de Ciencia y Tecnología, UACM.
3. Aho, A. V., Lam, M. S., Sethi, R., y Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2nd ed.). Pearson.
4. Vilar Torres, J. M. (2010). *Analizador léxico*. Universitat Jaume I.
5. Soto Romero, M. (2021). *Autómatas finitos* (Nota de clase 3, Teoría de la Computación). Colegio de Ciencia y Tecnología, UACM.
6. Soto Romero, M. (2021). *Autómatas finitos con ε -transiciones* (Nota de clase 5, Teoría de la Computación). Colegio de Ciencia y Tecnología, UACM.